Review 1 Agent 13 Secure Shredding Agent

Chapter 1: Introduction

- **1.1 Project Overview**
    - Agent 13 is the Secure Shredding Agent in our 35-agent Agentic Shell class project. This review presents its problem, objectives, scope and theoretical background. Our current deliverable is a standalone Ubuntu Linux service that overwrites an eligible file with random bytes, synchronizes it and removes its directory entry. It operates independently of a master shell or language model.
    - Traditional command line interfaces provide direct access to operating-system utilities, but users must learn command names, flags and path rules. The wider class project proposes natural-language interaction with specialized agents. Our group first implements the Linux capability and a stable interface; natural-language routing belongs to a later class integration phase.
    - The present interaction is explicit: the user runs agent13ctl, the CLI sends JSON over a Unix domain socket, and the background service calls a C shared library through Python ctypes. This division lets us demonstrate real Linux operations now and retain the same service when a common Agentic Shell becomes available.
- **1.2 Problem Statement**
    - Ordinary deletion removes a file name without deliberately overwriting its logical contents. A user may therefore confuse successful removal with complete erasure. At the same time, an incorrect path or destructive command can remove unintended data. Secure deletion needs both a controlled overwrite operation and checks that identify exactly which file the user approved.
    - A check performed only on a pathname is insufficient because the named file can change before execution. Symlinks, hardlinks, file replacement and unsupported file types introduce additional risks. Our problem is to provide a readable, testable service that confines deletion to a dedicated directory, validates the opened file and requires a fresh confirmation before mutation.

- **1.3 Objectives**
  - Implement an independent headless service using Python 3, a C11 shared library and real Linux/POSIX operations. The native sequence must overwrite one random pass, call fsync and unlink the staged file, with explicit error and partial-progress reporting.
  - Provide CLI health, information, inspection, preview and confirmed execution commands. Preview must preserve file contents and return an expiring, single-use token tied to the path and file fingerprint.
  - Reject traversal, symlinks, directories, multiple links, unsafe permissions, changed files and callers with another UID. Test all destructive behavior only with disposable files in temporary directories.
  - Prepare a versioned Unix socket API, Python-callable wrapper, documented C header and proposed tool manifest for later integration. Building an NL-to-CLI engine or MCP server is a future class task; it is not an objective of the current standalone agent.
- **1.4 Scope and Limitations**
  - The target is Ubuntu/Debian Linux with Python 3.10 or later, GCC, Make and systemd user services. Agent 13 runs as an ordinary user and exposes no GUI, web interface or network listener. Supported targets are owned, readable and writable regular files with one hardlink, located inside an explicitly configured private allowed directory.
  - The current scope includes service lifecycle, local IPC, validation, confirmation, native shredding, structured JSON responses, logging, automated tests and documentation. Only Agent 13 is implemented. The master shell, central registry, other 34 agents, LLM provider, intent router and MCP adapter are outside Phase A.
  - Root execution is refused, and no administrative privileges are delegated to an AI model. Socket permissions and Linux peer credentials restrict service access to the current UID. A malicious process sharing that UID, root and writers that ignore advisory locks remain outside the trust boundary.

- Overwriting cannot guarantee physical erasure on SSDs, copy-on-write storage, snapshots, backups or devices with remapped sectors. The operation covers the current logical file size, not historical copies or filename metadata. A successful result demonstrates best-effort overwrite and unlink behavior, not certified media sanitization [1].

Chapter 2: Literature Review & Theoretical Background

- **2.1 Large Language Models (LLMs) in Code Generation**
    - Large language models can generate code or structured text from natural-language descriptions. Chen et al. studied code generation using executable HumanEval tests and also reported limitations in following complex specifications [2]. This work supports evaluating generated programs through actual behavior rather than judging correctness from plausible text alone.
    - For a future Agentic Shell, a model could propose a tool name and arguments. A plausible but incorrect tool choice, invented option or misunderstood path could still be destructive. Our design therefore keeps authorization, path validation and confirmation in deterministic service code. Model output would be treated as an untrusted request; Phase A uses no LLM.
- **2.2 Introduction to Model Context Protocol (MCP)**
    - The Model Context Protocol describes a host application, clients that connect to servers, and server-provided capabilities such as tools. Named tools with structured inputs can help an AI application discover and invoke external functions through a common protocol. MCP concerns context and tool exchange; it does not itself decide application intent or establish that a destructive action is safe [3].
    - A common tool protocol may reduce custom adapter work when the class later connects many agents. Its advantage is interoperability, not automatic permission to access local files. Agent 13 currently uses its own small versioned JSON contract over a Unix socket. It is not an MCP server, and its JSON messages are not MCP JSON-RPC messages.
    - The proposed manifest names health, inspect_target, preview_shred and execute_shred. A future adapter could translate tool calls to this existing service while preserving its checks. The class must still agree on the common contract and human approval workflow before integration; neither the adapter nor the master dispatcher is implemented.

- **2.3 Existing Solutions**
    - GitHub Copilot CLI provides AI-assisted work in a terminal, and Warp provides conversational agent interactions within its terminal environment [4, 5]. These illustrate natural-language interfaces for developer tasks. They are broader tools than our assignment and are not dependencies of Agent 13; their features do not establish a physical-erasure guarantee.
    - GNU Coreutils shred is an open-source file-overwriting utility and a relevant functional reference. Its manual explains the assumption that writes replace data in place and the limitations caused by storage layers and retained copies [1]. Agent 13 instead implements a small C backend so the project demonstrates descriptors, random-byte generation, complete write handling, fsync and unlink directly.
    - The resulting design combines a narrow operation with a service boundary, confined targets and a preview token workflow. It does not claim to improve the physical erasure properties of the underlying storage. Its contribution to the class project is an independently demonstrable Linux capability with interfaces ready for later adaptation.

References

- [1] GNU Coreutils. shred invocation. https://www.gnu.org/s/coreutils/manual/html_node/shred-invocation.html
- [2] Chen M et al. Evaluating Large Language Models Trained on Code. 2021. https://arxiv.org/abs/2107.03374
- [3] Model Context Protocol. Architecture overview. https://modelcontextprotocol.io/docs/learn/architecture
- [4] GitHub. GitHub Copilot CLI. https://github.com/github/copilot-cli
- [5] Warp Documentation. Interacting with agents. https://docs.warp.dev/agents/local-agents/interacting-with-agents/
- Project sources: class Project-Overview; Agent 13 README.md, docs/ROADMAP.md, docs/SECURITY.md and docs/INTEGRATION.md. Online references accessed 15 October 2026.